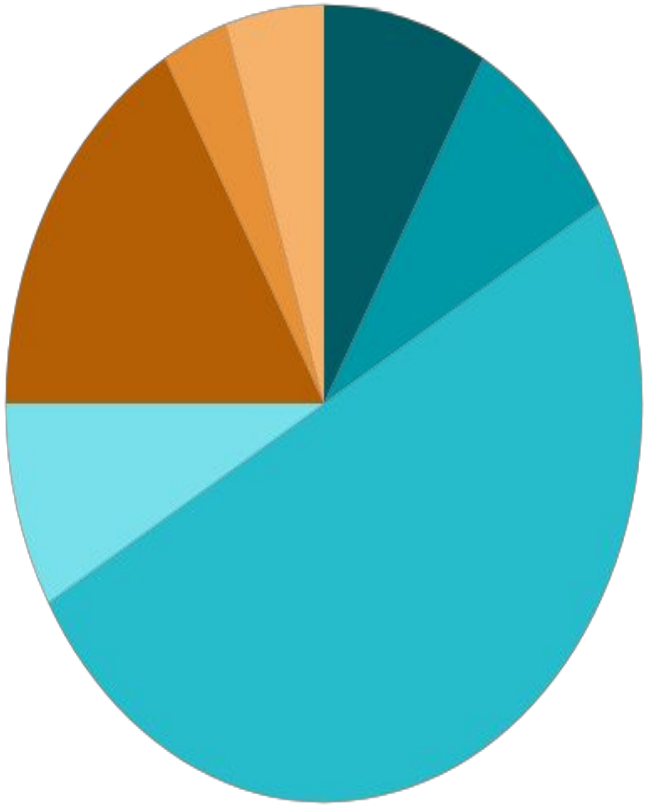
GSFC UNIVERSITY
EDUCATION RE-ENVISIONED

# Unit 3 - Generative Adversarial Networks (GANs)

# Outline

- Introduction to Deep Learning for Generative AI Neural Network Architectures
- Review of Autoencoders
- Variational Autoencoders (VAEs)
- Loss Functions in Generative Models Summary and Discussion

# Lecture Structure



| | |
|---|---|
| 05 Minutes | Outline & Objectives |
| 05 Minutes | Recap of Previous Lecture |
| 30 Minutes | Delivery of Actual Content as per Lesson Plan |
| 05 Minutes | Industry Relevance |
| 10 Minutes | Interactive Activities |
| 02 Minutes | Importance of Discipline |
| 03 Minutes | Recording of Attendance |

# Introduction to GANs

**What is a GAN?**

**GAN stands for Generative Adversarial Network.**

A GAN is a type of **Generative AI model** that learns to create new data that looks similar to the data used for training.

For example:

Give GAN thousands of human face images → it can generate **new artificial faces**.

Give it paintings → it can generate **new paintings**.

Give it handwritten digits → it can generate **new digits**.

Give it photographs → it can generate realistic-looking photographs.

**A GAN is a deep learning model in which two neural networks compete with each other to generate realistic data.**

The two networks are:

**Generator (G)**

**Discriminator (D)**

Think of it as a competition between a **counterfeiter and a detective**.

# Basic Idea Behind GAN

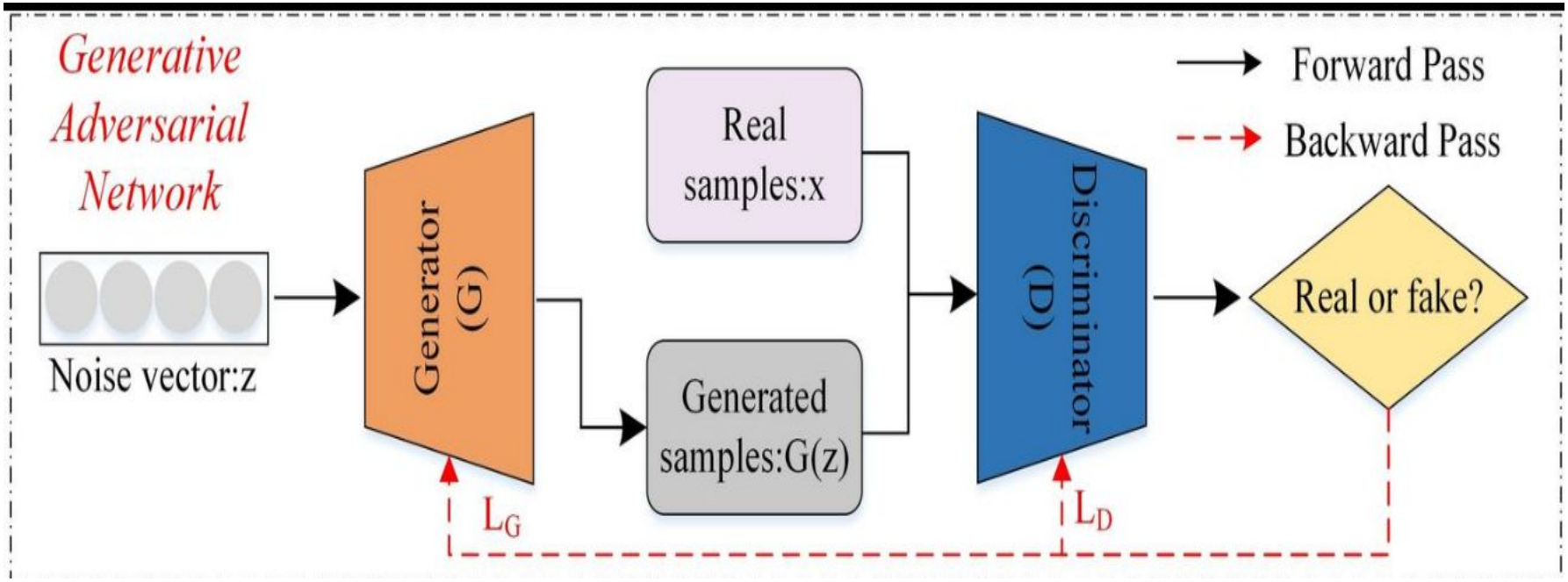
Suppose we want to generate handwritten digits.
We give the GAN thousands of images of handwritten digits.
The generator produces:

**Generator**
↓
**Fake digit**
↓
**Discriminator**
↓
**Real or Fake**?

**Beginning: Generator → ugly/random digit → Discriminator → FAKE**

**After training : Generator → realistic digit → Discriminator → REAL**

# Architecture of GAN



**A. Generator**

The **Generator (G)** creates new samples.

It takes a random input called **noise** or **latent vector**.

For example:

Random Noise

↓

Generator

↓

Generated Image

The noise can be represented as:

$z \sim Pz(z)$

where:

- z = random noise
- Pz = probability distribution

For example:

$$z = [0.21, -0.72, 0.45, 0.91, ...]$$

The generator transforms this random vector into an image:

$G(z)$

**Important point**

The generator does **not simply copy an existing image**.

It learns the underlying patterns of the training data and tries to create a **new sample**.

# Discriminator

The **Discriminator (D)** is a binary classifier.
It receives two types of inputs:
**Real data**
Data directly obtained from the training dataset.
**Fake data**
Data generated by the Generator.
The discriminator produces a probability:
D(x)
where the output is generally between **0 and 1**.
For example:

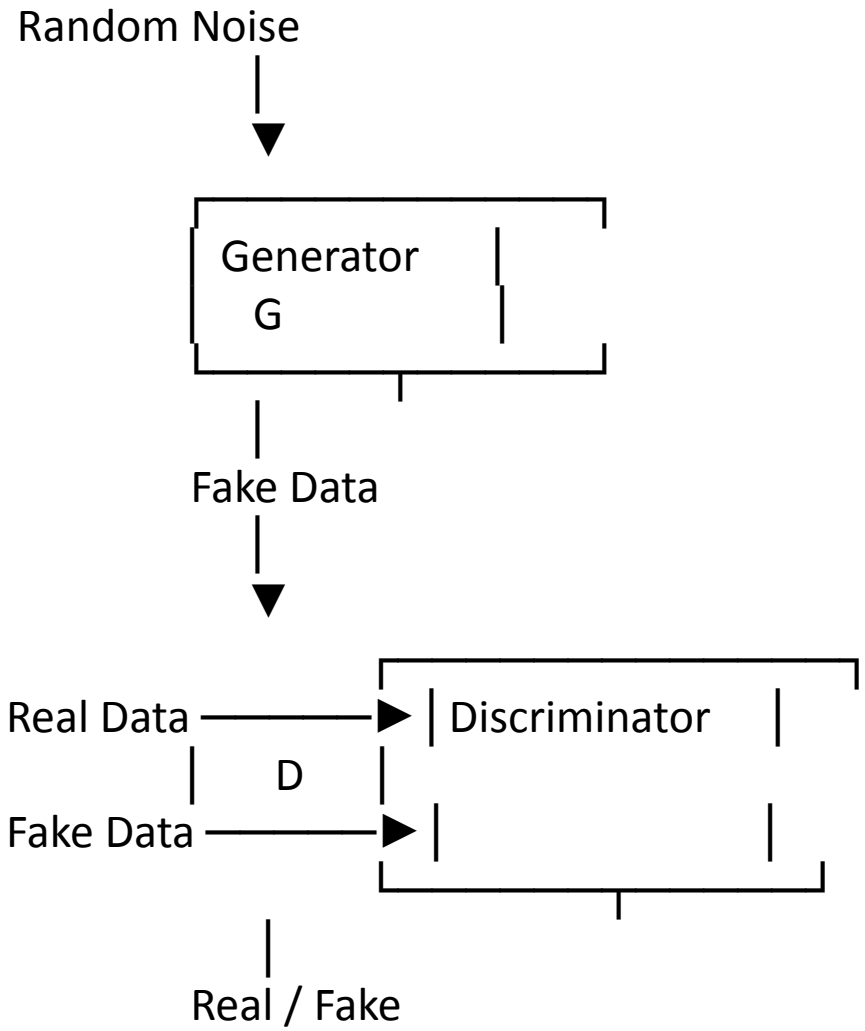
Real Image → Discriminator → 0.95 → REAL
Fake Image → Discriminator → 0.08 → FAKE

A value close to:

- **1 → Real**
- **0 → Fake**

**Complete GAN Architecture**

The complete process is:



**What happens?**

The discriminator gets:

**Real data + Fake data**

and learns to distinguish between them.

The generator gets feedback from the discriminator and learns to make better fake data.

# GAN Workflow

**Step 1: Collect real data**
Suppose our dataset contains:
**10,000 images of human faces**

Training Dataset
↓
Real Face Images

**Step 2: Generate random noise**
We start with random numbers.

Random Noise
z = [0.2, 0.7, -0.4, 0.9, ...]

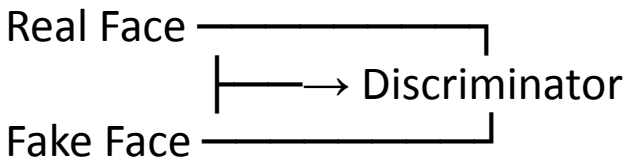
**Step 3: Generator creates fake data**
The generator takes the noise and produces an artificial face.

Random Noise
↓
Generator
↓
Fake Face

Initially, the face may look terrible.

**Step 4: Discriminator receives real and fake data**

The discriminator receives:




It predicts:

```
Real → 1
Fake → 0
```

**Step 5: Calculate loss**

The discriminator calculates how well it classified the images.
The generator also receives feedback.
If the discriminator easily detects the fake image:
Generator needs improvement.

**Step 6: Update the networks**

Using **backpropagation and gradient descent**:

- Generator parameters are updated.
- Discriminator parameters are updated.

This process repeats many times.

**Step 7: Generator becomes better**

After many iterations:

**Bad Fake Image**

↓

**Training**

↓

**Better Fake Image**

↓

**Training**

↓

**Very Realistic Fake Image**

**Eventually:**

**Generator → Highly realistic images**

**Discriminator → Difficult to distinguish real/fake**

# Why is it called "Adversarial"?

The word **adversarial** means **opposing or competing**.
The two networks have opposite objectives.
**Generator's objective**
"I want to fool the discriminator."
**Discriminator's objective**
"I want to correctly identify fake data."
Therefore:

Generator
↓
Tries to fool
↓
Discriminator
↓
Tries to detect
↓
Generator improves

This competition gives GANs their name:
**Generative + Adversarial + Network**

# Step 8. GAN Objective Function

The original GAN uses a **minimax objective**:

GminDmaxV(D,G)

where:

- G = Generator
- D = Discriminator

The objective is:

$V(D,G)=E_{x\sim p_{data}(x)}[\log D(x)]+E_{z\sim p_z(z)}[\log(1-D(G(z)))]$

Don't worry about making students memorize this initially.

Explain the meaning:

**Discriminator wants:**

For real data:

$D(x)\rightarrow 1$

For generated data:

$D(G(z))\rightarrow 0$

**Generator wants:**

$D(G(z))\rightarrow 1$

because the generator wants the discriminator to believe that fake data is real.

# 02

**DCGAN · CGAN · CycleGAN · StyleGAN**

# Types of GANs

Your syllabus contains four important GAN architectures:

1. **DCGAN**
2. **CGAN**
3. **CycleGAN**
4. **StyleGAN**

Let's understand each one.

**Generative Adversarial Network (GAN)** is a deep learning framework used to generate new data that resembles real training data.

A GAN consists of two neural networks:

- **Generator (G)**
  - Creates synthetic/fake samples
  - Takes random noise as input
  - Attempts to generate realistic data
- **Discriminator (D)**
  - Receives real and generated samples
  - Determines whether a sample is real or fake

**Basic idea**

**Random Noise → Generator → Fake Image**

**Real Image + Fake Image → Discriminator → Real/Fake**

# DCGAN

**DCGAN = Deep Convolutional GAN**

DCGAN stands for:

**Deep Convolutional Generative Adversarial Network**

It combines GAN architecture with **Convolutional Neural Networks (CNNs).**

The original GAN architecture can use fully connected layers, but images are better handled by CNN-based architectures.

**Key idea:**

GAN + CNN architecture = DCGAN

Instead of using only fully connected layers, DCGAN uses convolutional operations to learn spatial features from images.

DCGAN

## DCGAN — Deep Convolutional GAN

- Proposed by Radford, Metz & Chintala (2015) as one of the first stable, convolution-based GAN architectures.
- Replaced the fully-connected layers of the original GAN with convolutional and transposed-convolutional layers.
- Established a widely-adopted set of architectural guidelines that dramatically improved training stability.
- Demonstrated that the learned latent space carries meaningful, semantically smooth structure (e.g. vector arithmetic on faces).
- Became the de-facto baseline architecture for most image-based GANs that followed.

## Why DCGAN Was Needed

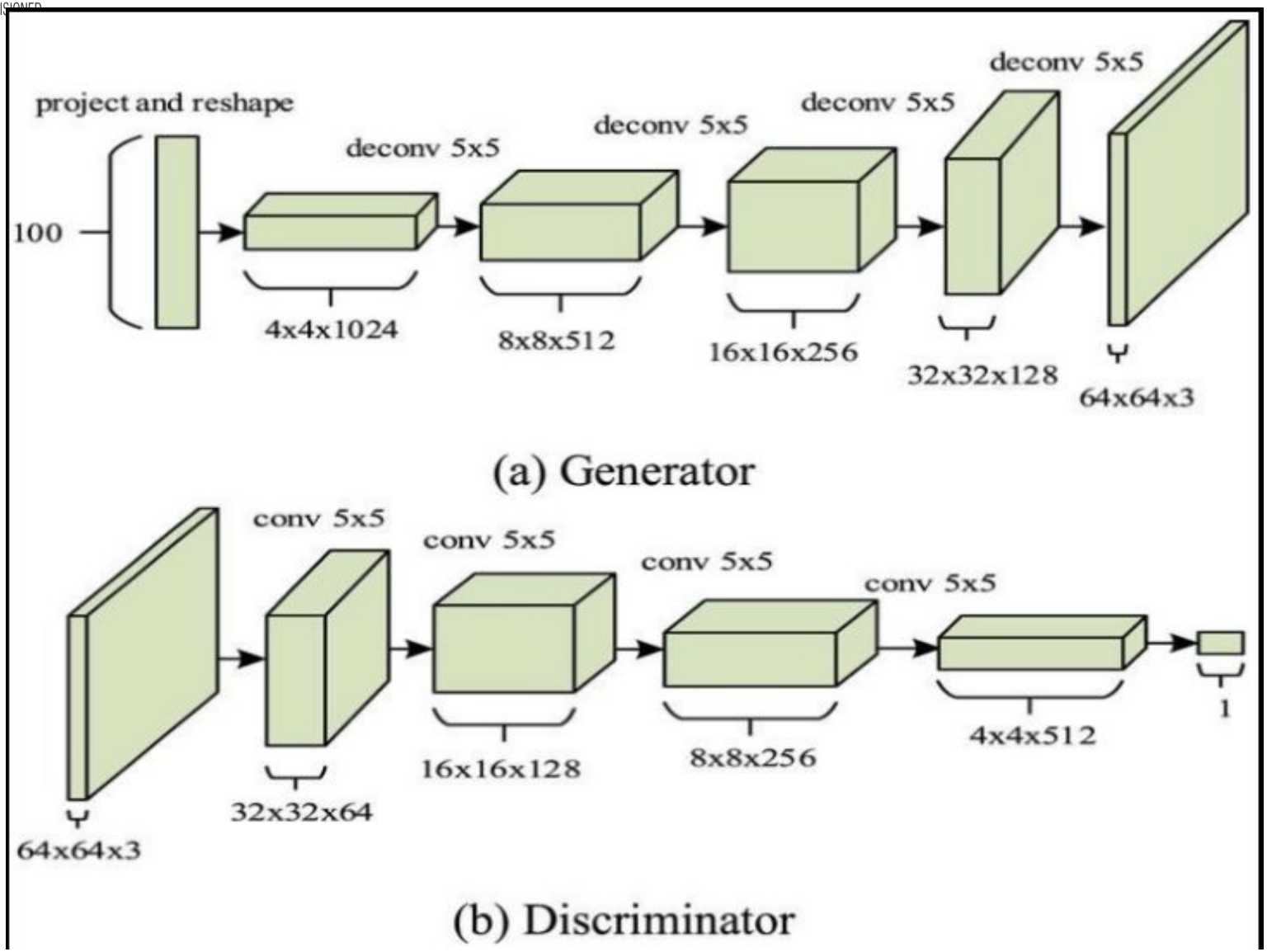
- Original GANs used stacked fully-connected (dense) layers — very high parameter count and poor at capturing spatial structure in images.
- Fully-connected GANs were highly prone to instability, mode collapse, and poor sample quality on anything beyond MNIST-scale data.
- Convolutional layers naturally exploit spatial locality and parameter sharing — ideal for image data.
- DCGAN's contribution wasn't a new loss function — it was a set of empirically validated architecture design choices that made conv-GANs trainable.

**DCGAN solution**

CNNs are naturally suitable for images because they learn:

**Edges** → **textures** → **shapes** → **objects**

Therefore, DCGAN can learn spatial features more effectively.


project and reshape
deconv 5x5
deconv 5x5
deconv 5x5
deconv 5x5
100
4x4x1024
8x8x512
16x16x256
32x32x128
64x64x3
(a) Generator
conv 5x5
conv 5x5
conv 5x5
conv 5x5
64x64x3
32x32x64
16x16x128
8x8x256
4x4x512
1
(b) Discriminator

**Generator**

z⟶FC/reshape⟶ConvTranspose⟶ConvTranspose⟶Image

Random noise is gradually transformed into an image.

**Discriminator**

Image⟶Conv⟶Conv⟶Conv⟶Probability

The discriminator gradually extracts visual features and predicts whether the image is real or fake.

# DCGAN Generator

**Input**

Random noise vector:

$z \sim N(0, I)$

Example:

**100-dimensional noise vector**

**Process**

Noise → Feature representation → Upsampling → Image

Typical components:

- Transposed convolution
- Batch normalization
- ReLU activation
- Tanh at output

**Important concept**

The generator progressively increases spatial resolution.

Example:

**4×4 → 8×8 → 16×16 → 32×32 → 64×64**

# DCGAN Discriminator

The discriminator performs the reverse operation.

**Process**

**Image → Convolution → Feature extraction → Classification**

Typical components:

- Convolution layers
- Batch normalization
- Leaky ReLU
- Sigmoid/output probability

Example:

**64×64 → 32×32 → 16×16 → 8×8 → 4×4 → Real/Fake**

# DCGAN Training Workflow

- Generate random noise.
- Generator creates fake image.
- Obtain real image from dataset.
- Give real and fake images to discriminator.
- Discriminator predicts real/fake.
- Calculate discriminator loss.
- Update discriminator.
- Generate another fake image.
- Update generator based on discriminator feedback.

Repeat.

**Main learning loop**

**Generate → Discriminate → Calculate Loss → Update → Repeat**

| Guideline | Original GAN | DCGAN |
|---|---|---|
| Downsampling / Upsampling | Pooling layers | Strided / fractional-strided convolutions |
| Hidden layers | Fully-connected | Fully convolutional |
| Normalization | None | Batch normalization in G & D |
| Generator activation | ReLU / sigmoid mix | ReLU (Tanh at output layer) |
| Discriminator activation | Sigmoid-heavy | LeakyReLU (slope 0.2) throughout |

# DCGAN Advantages and Limitations

**Advantages**

- Better image generation than basic GAN
- CNN-based architecture
- Learns spatial features effectively
- Useful for image datasets
- More stable training than early GANs

**Limitations**

- Training can still be unstable
- Mode collapse can occur
- Limited control over generated images
- Difficult to generate highly detailed images
- Requires careful hyperparameter selection

# DCGAN Applications

## Applications

- Synthetic image generation
- Face generation
- Data augmentation
- Image representation learning
- Feature learning

## Example

Training on face images:

**Noise → DCGAN → New synthetic face**

The generated face does not correspond to a real person but resembles the learned distribution.

# CGAN = Conditional Generative Adversarial Network

CGAN introduces **additional information/conditions** into the GAN.
Instead of asking:
“Generate anything.”
We can ask:
“Generate something according to this condition.”

**Core idea**
**GAN + Condition = CGAN**

# Why Do We Need CGAN?

A traditional GAN does not provide precise control over what it generates.

For example:

A GAN trained on:

**Cat + Dog + Car + Bird**

may generate any of these.

But we may want:

“Generate a cat.” CGAN provides this control.

**Condition can be:**

Class label

Text

Attribute

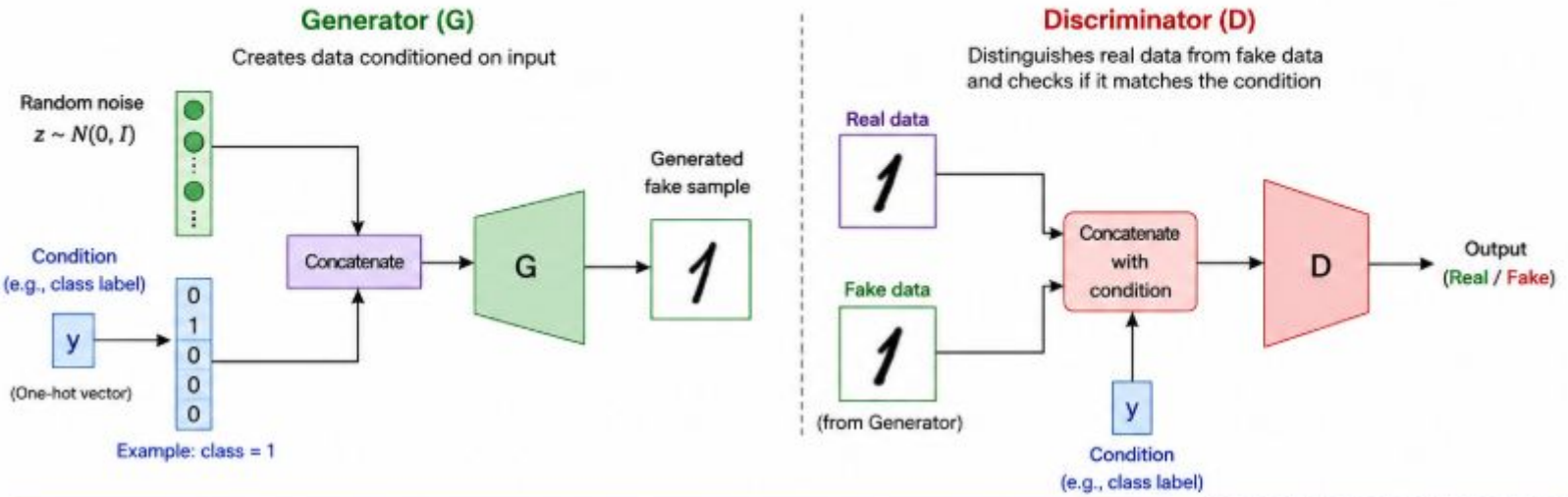
Image

Category

Other metadata

# Conditional GAN (CGAN)

CGAN generates data conditioned on additional information.




**Training Process**

**Step 1: Update Discriminator (D)**

- Provide real data + condition → label = Real (1) — 1
- Provide fake data + condition → label = Fake (0) — 1
- D tries to correctly classify real or fake
- Update D to minimize classification loss

**Step 2: Update Generator (G)**

- Generator creates fake data using noise + condition
- Discriminator evaluates (fake data + condition)
- G tries to fool D (make D predict Real)
- Update G to minimize generator loss

**Example (Digit Generation)**

Condition (y): class label (0-9)

Generator generates the digit corresponding to the given class label.

Example:

y = 0 1 0 0 0 0 0 0 0 0

Generated digit → 1

**Key Points**

- Both Generator and Discriminator receive the condition.
- Generator learns to produce data that matches the condition.
- Discriminator checks both authenticity and condition consistency.
- CGAN provides more control over data generation.
- Widely used for image generation, data synthesis, and more.

CGAN provides the condition to **both Generator and Discriminator**.

**Generator**

G(z,y)

where:

- z = random noise
- y = condition

**Discriminator**

D(x,y)

where:

- x = image
- y = condition

# CGAN Workflow

**Step 1**
Select condition:
**Label = Cat**
**Step 2**
Generate random noise.
**Step 3**
Generator receives:
**Noise + Cat label**
**Step 4**
Generator creates cat image.
**Step 5**
Discriminator receives:
**Image + Cat label**
**Step 6**
Discriminator determines whether it is a realistic cat.
**Step 7**
Generator learns from discriminator feedback.

# CGAN Advantages and Limitations

## Advantages

- Controlled generation
- Class-specific image generation
- Useful for data augmentation
- Better control than basic GAN

## Limitations

- Requires labeled/conditional information
- Condition may not always be correctly represented
- Training can still be unstable
- Quality depends on the quality of conditioning information

# CGAN Applications

## Applications

- Handwritten digit generation
- Class-specific image synthesis
- Face generation with attributes
- Image-to-image generation
- Data augmentation
- Controlled text/image generation

## Example

**Condition = “Young + Male + Smiling”**

⟶ Generate face satisfying these conditions.

# CycleGAN = Cycle-Consistent Generative Adversarial Network

- It performs **image-to-image translation without requiring paired images**.

For example:

**Horse → Zebra**

**Zebra → Horse**

**Summer → Winter**

**Winter → Summer**

The important feature is:

Images do NOT need to be paired.

# The Problem with Traditional Image-to-Image Translation

| Horse | Corresponding Zebra |
| --- | --- |
| Horse A | Zebra A |
| Horse B | Zebra B |
| Horse C | Zebra C |

These are called **paired datasets**.
Creating such pairs is difficult because the two images must represent corresponding scenes.
CycleGAN removes this requirement.

# CycleGAN Uses Unpaired Data

Instead of paired data, we have two independent datasets:

**Domain X**

- Horse images
- Horse 1
- Horse 2
- Horse 3

**Domain Y**

- Zebra images
- Zebra 1
- Zebra 2
- Zebra 3

There is **no requirement that Horse 1 corresponds to Zebra 1**.

This is called:

**Unpaired Image-to-Image Translation**

# Basic CycleGAN Architecture

CycleGAN contains:

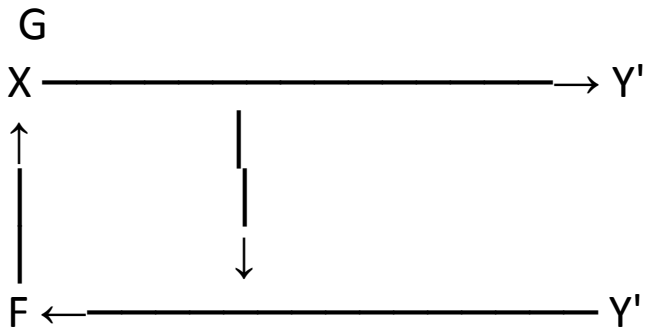
**Two Generators**

- G: X ⟶ Y
- F: Y ⟶ X

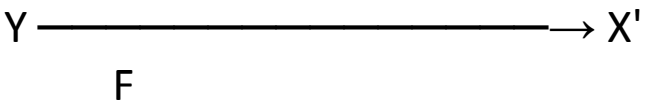
**Two Discriminators**

- $D_Y$: distinguishes real Y from generated Y
- $D_X$: distinguishes real X from generated X

Architecture:



And the reverse:




The key idea is **cycle consistency**.

# Why Two Generators?

Generator **G** learns:

**X → Y**

Example:

**Horse → Zebra**

Generator **F** learns:

**Y → X**

Example:

**Zebra → Horse**

Having two generators allows the model to learn translation in **both directions**.

# Why Two Discriminators?

Each domain needs its own discriminator.

**$D_Y$**

Checks whether an image looks like a real **Y-domain image**.

Example:

**Real Zebra → Real**

**Generated Zebra → Fake**

**$D_X$**

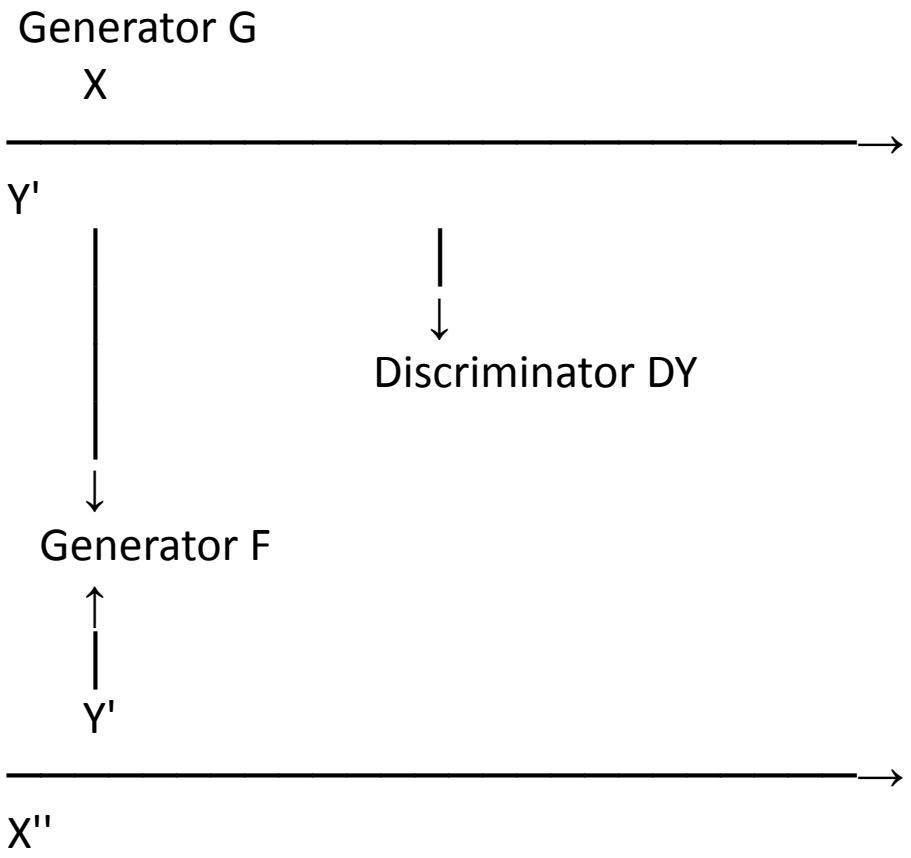
Checks whether an image looks like a real **X-domain image**.

Example:

**Real Horse → Real**

**Generated Horse → Fake**

# Complete CycleGAN Architecture



# Complete CycleGAN Architecture

**Reverse cycle:**

$$Y \longrightarrow F(Y) \longrightarrow X' \longrightarrow G(X') \longrightarrow Y''$$

The model wants:
**$X \approx F(G(X))$**
and
**$Y \approx G(F(Y))$**

# Cycle Consistency

This is the most important concept in CycleGAN.
Suppose:
**Horse → Zebra**
Then we take the generated zebra and convert it back:
**Zebra → Horse**
Ideally:
**Original Horse ≈ Reconstructed Horse**
Mathematically:
**F(G(X)) ≈ X**
Similarly:
**G(F(Y)) ≈ Y**
This is called:
**Cycle Consistency**

# Simple Example of Cycle Consistency

Input:

 **Horse**

↓

Generator G

↓

 **Generated Zebra**

↓

Generator F

↓

 **Reconstructed Horse**

The reconstructed horse should retain important characteristics of the original horse.

# Why Cycle Consistency is Needed?

Without cycle consistency, the generator could produce arbitrary images that simply fool the discriminator.

For example:

**Horse → Random Zebra**

Even if the discriminator accepts it, the important structure of the original image may be lost.

Cycle consistency encourages:

- Content preservation
- Structural consistency
- Meaningful translation

# CycleGAN Loss Functions

CycleGAN uses several losses.

**1. Adversarial Loss**

Makes generated images look realistic.

**2. Cycle Consistency Loss**

Ensures:

**X → Y → X**

returns approximately the original X.

**3. Identity Loss**

Helps preserve the image when it is already in the target domain.

# Adversarial Loss

For generator G:

**G: X → Y**

The discriminator $D_Y$ tries to distinguish:

- Real Y
- Generated G(X)

The generator tries to make:

**G(X) ≈ Real Y**

Similarly, F tries to generate realistic X-domain images.

# Cycle Consistency Loss

The cycle consistency loss is commonly written as:

$$\mathbf{L_{cycle}(G,F) = E[||F(G(x)) - x||_1] + E[||G(F(y)) - y||_1]}$$

Don't worry about memorizing the equation initially.

The basic meaning is:

After translating an image to another domain and translating it back, the reconstructed image should be close to the original.

# Identity Loss

Suppose an image is already from domain Y.
Ideally:
**G(Y) ≈ Y**
Similarly:
**F(X) ≈ X**

Identity loss helps prevent unnecessary changes.
For example, if a generator receives an image that already belongs to the target style, it should avoid changing it dramatically.

# Overall CycleGAN Objective

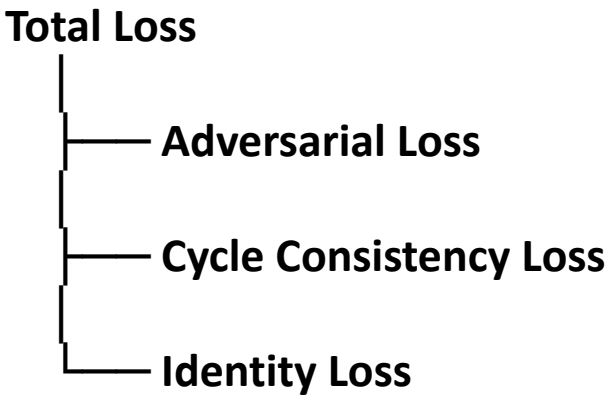
The total objective combines:

**Adversarial Loss**

**Cycle Consistency Loss**

**Identity Loss**

Conceptually:



# CycleGAN Training Process

**Step 1:** Take image X.

**Step 2:** Generate Y' using G.

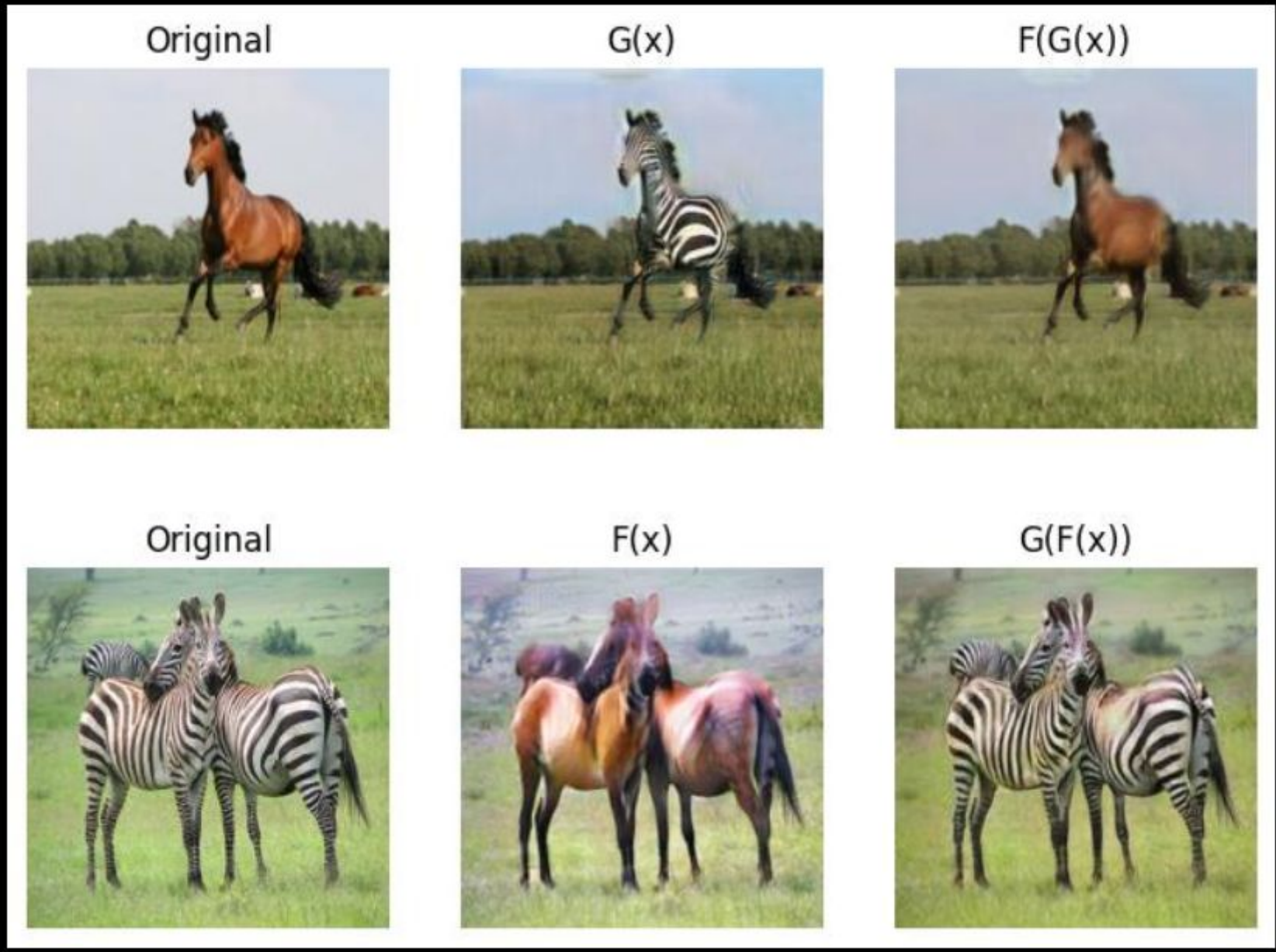
**Step 3:** $D_Y$ checks whether Y' looks real.

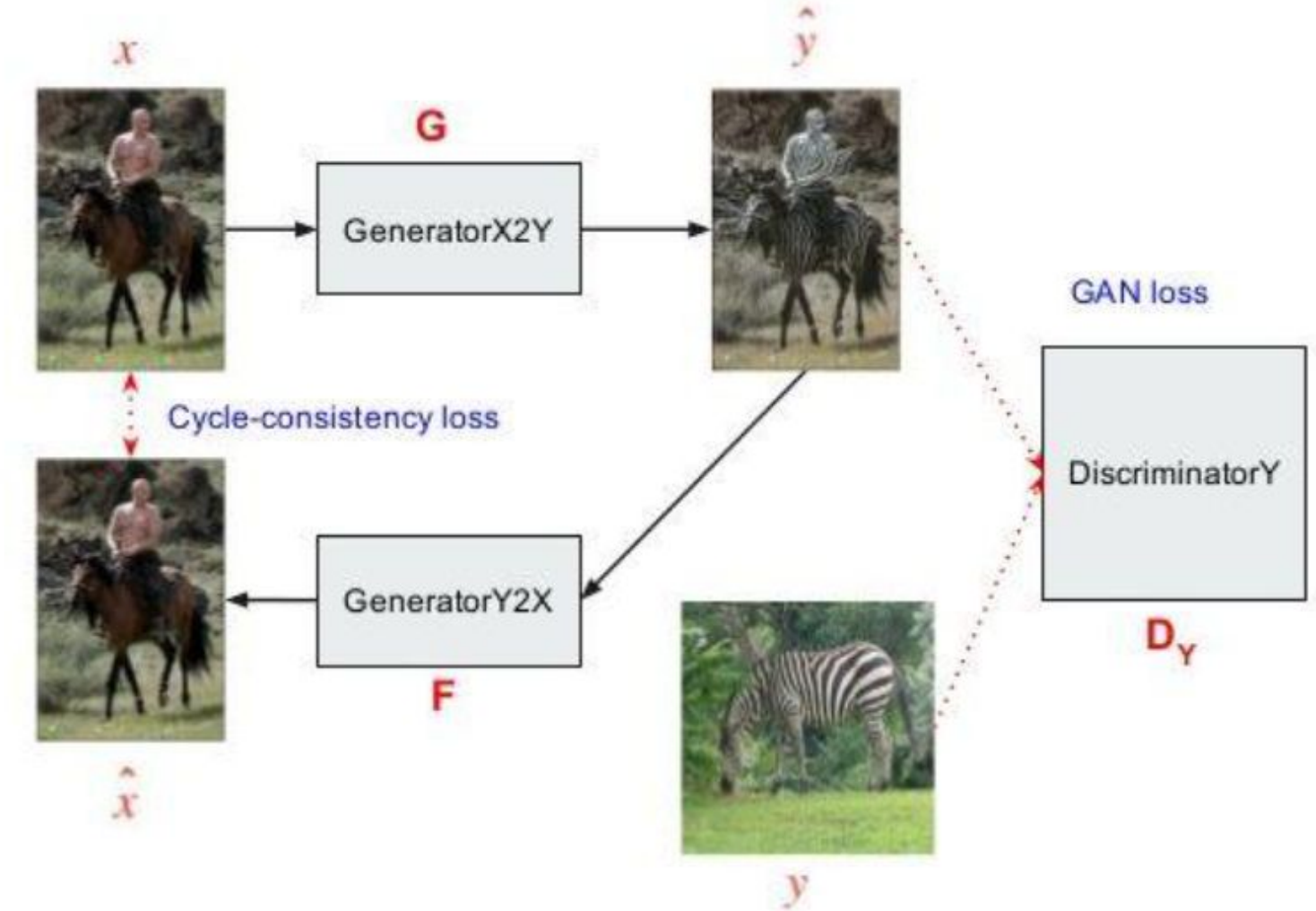
**Step 4:** Convert Y' back to X'' using F.

**Step 5:** Compare X'' with original X.

**Step 6:** Repeat the process in the opposite direction.

**Step 7:** Update generators and discriminators.


Original
G(x)
F(G(x))
Original
F(x)
G(F(x))


x
G
GeneratorX2Y
ŷ
GAN loss
Cycle-consistency loss
DiscriminatorY
D_Y
GeneratorY2X
F
x̂
y

Input Image

Predicted Image

**Input:** Horse image

**Output:** Zebra-like image

The model learns visual characteristics such as:

- Stripes
- Texture
- Color patterns

while attempting to preserve:

- Shape
- Pose
- Background
- Spatial structure

# Other CycleGAN Applications

**CycleGAN can be used for:**

- Horse ↔ Zebra
- Summer ↔ Winter
- Monet ↔ Photograph
- Photo ↔ Painting
- Day ↔ Night
- Apple ↔ Orange
- Synthetic ↔ Real images
- Domain adaptation

# Advantages of CycleGAN

**Advantages**

- Does not require paired datasets
- Useful for domain translation
- Learns two-way mappings
- Preserves important structural information
- Useful when paired data is difficult to obtain

# Limitations of CycleGAN

- Training can be unstable
- Results depend heavily on training data
- May introduce unwanted artifacts
- Not ideal for every type of translation
- Can struggle with large geometric changes
- It may learn shortcuts instead of true semantic transformations

# StyleGAN

StyleGAN = Style-based Generative Adversarial Network

Developed by NVIDIA researchers.
The main goal is to generate:
**High-quality, realistic and controllable images.**
It became particularly famous for generating highly realistic **human faces**.

# Why StyleGAN?

Traditional GANs generally use:

**Random Noise → Generator → Image**

But controlling the generated image can be difficult.

For example, we may want to independently control:

- Face shape
- Hair
- Age
- Pose
- Skin appearance
- Facial expression
- Clothing

StyleGAN introduces a more controllable generation process.

# Traditional GAN vs StyleGAN

**Traditional GAN**

Random Noise

↓

Generator

↓

Image

**StyleGAN**

Random Noise

↓

Mapping Network

↓

Intermediate Latent Space

↓

Style Modulation

↓

Synthesis Network

↓

Image

# StyleGAN Architecture

StyleGAN contains two major components:

**1. Mapping Network**

Transforms:

**z ⟶ w**

**2. Synthesis Network**

Transforms:

**w ⟶ Image**

The intermediate latent vector **w** controls the styles applied during image generation.

# Mapping Network

The input is a random latent vector:

**z**

Instead of directly feeding z into the generator, StyleGAN passes it through a mapping network.

z

↓

Fully Connected Layers

↓

w

Typically, the mapping network consists of multiple fully connected layers.

The purpose is to transform the original latent space into a more useful **intermediate latent space**.

# Why Intermediate Latent Space?

The original latent space **Z** may contain entangled features. For example, changing one dimension could unintentionally change:

- Hair
- Face shape
- Age

StyleGAN creates another space:

**W Space**

which provides better control over visual attributes.

# Synthesis Network

The synthesis network converts the intermediate latent representation into an image.

Constant Input
↓
Style Modulation
↓
Convolution
↓
Upsampling
↓
Convolution
↓
Upsampling
↓
High Resolution Image

The network gradually increases image resolution.

# Style Modulation

This is one of the most important concepts in StyleGAN.
The latent vector **w** controls the style of each layer.
For example:

**Early layers**

Control coarse features:

- Pose
- Face shape
- Head orientation

**Middle layers**

Control:

- Hair
- Facial features
- Texture

**Later layers**

Control fine details:

- Skin texture
- Hair strands
- Small visual details

# Adaptive Instance Normalization (AdaIN)

The original StyleGAN uses:

**Adaptive Instance Normalization — AdaIN**

The style vector controls the feature statistics.

Conceptually:

Feature Map

↓

Normalize

↓

Scale + Bias

↑

Style

The style determines how the feature map is modified.

# StyleGAN Layer-by-Layer Control

| Network Level | Main Information |
| --- | --- |
| Early | Overall structure |
| Middle | Object/face features |
| Late | Fine details |

Therefore:
**StyleGAN provides hierarchical control over generated images.**

# Noise Injection

StyleGAN also introduces **noise** at different layers.

Why?

Because some details are random and should not be completely controlled by the latent vector.

Examples:

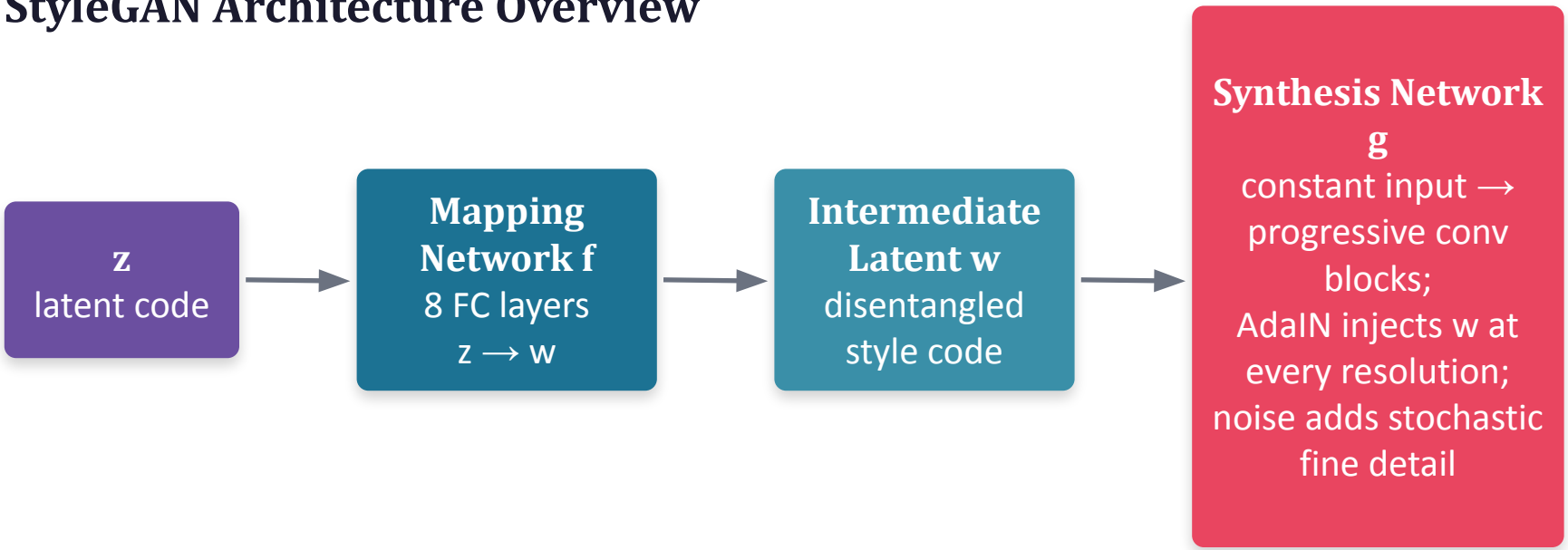
- Skin pores
- Hair variations
- Small texture details
- Random facial details

So:

**Style → Controls structured features**

**Noise → Controls stochastic fine details**

# StyleGAN Architecture Overview




*The Mapping Network first untangles z into a more disentangled style code w. The Synthesis Network then starts from a learned constant and repeatedly applies AdaIN using w at every resolution, with independent noise injected for stochastic detail.*

# StyleGAN Generation Example

Random z
↓
Mapping Network
↓
w
↓
Style Information
↓
Synthesis Network
↓
Realistic Face

StyleGAN can combine styles from different latent vectors.
For example:
**Person A**
⟶ Face structure
**Person B**
⟶ Hair style
The network can combine these characteristics.
This is called:
**Style Mixing**

# Style Mixing Example

Latent A
↓
Early Layers
↓
Face Structure

Latent B
↓
Later Layers
↓
Hair / Texture

Result:
**New image combining characteristics of A and B**
This demonstrates the controllability of the StyleGAN latent space.

A generated image can be modified by moving in the latent space.

Latent Vector

↓

+ Age Direction

↓

Older Face

or:

Latent Vector

↓

+ Smile Direction

↓

Smiling Face

This makes StyleGAN useful for **image editing and manipulation**.

# StyleGAN Advantages

## Advantages

- Extremely realistic images
- High-resolution generation
- Better control over image attributes
- Meaningful latent space
- Style mixing
- Useful for image editing
- Hierarchical feature control

# StyleGAN Limitations

- Computationally expensive
- Requires significant training resources
- Training can still be challenging
- Generated images can contain subtle artifacts
- Bias in training data can appear in generated outputs
- Primarily associated with image generation rather than general image translation

| Feature | CycleGAN | StyleGAN |
|---|---|---|
| Main Purpose | Image translation | Image generation |
| Data | Unpaired datasets | Image dataset |
| Generators | Two | One synthesis generator |
| Discriminators | Two | One |
| Main Concept | Cycle consistency | Style-based generation |
| Direction | X ↔ Y | Latent → Image |
| Major Strength | Domain translation | High-quality controllable images |
| Example | Horse → Zebra | Generate realistic face |

## In every lecture, Faculty may talk on any of the following:

- **DCS & GSFCU Portal:** Remind students to use DCS, talk about it's features and instruct all to visit GSFCU website
- **Discipline, Values and Ethics**.
- **GSFC University – My University:** Emphasize the importance of taking ownership of the University.
- **Respect & Courtesy:** Remind students to be respectful to peers, teachers, and staff. Encourage polite language and behavior.
- **Punctuality:** Emphasize the importance of arriving on time to class and completing assignments by deadlines.
- **Classroom Rules:** Reinforce basic classroom rules, such as keeping mobile phones on silent and maintaining a clean environment.
- **Safety:** Remind students about safety protocols, whether it's physical safety in the classroom or digital safety when using online resources.
- **Positive Attitude:** Encourage a positive mindset, cooperation and supporting one another.

# GSFCU & Values

Respect & Courtesy

**Font**
Type: Verdana
Size: 20 / 18 / 16
Colour: Black


The End